从 Copy GC 迁移到 Go 风格并发标记-清扫 GC：分步实施方案

文档目标

基于当前 Cangjie Runtime 的区域复制式 GC（Regional Copying GC），按步骤改造为类似 Go 的**并发三色标记-清扫（Concurrent Tri-color Mark-and-Sweep）** GC，以实现亚毫秒级 STW 停顿和更简洁的指针语义。

第一部分：架构对比与迁移动机

1.1 当前 Cangjie GC 架构（Regional Copying GC）

```
分配: RegionSpace (bump-pointer per-thread AllocBuffer)
          |
标记: Enum → Trace (并发, SATB写屏障, bitmap-on-region)
          |
搬移: Preforward → Forward (并行复制, CAS锁定, tagged-pointer自愈)
          |
回收: CollectSmallSpace → garbageRegionList → dirtyUnitTree → releasedUnitTree
```

核心特征：

- **移动式**：存活对象从 from-space 复制到 to-space，消除碎片
- **Tagged Pointer**：指针低位编码 tag ID（0/1 交替），用于区分"已转发"/"未转发"
- **自愈读写屏障**：读/写遇到 from-space 指针时触发延迟转发
- **6 套屏障实现**：Idle / Enum / Trace / PostTrace / Preforward / Forward
- **Phase 数量**：8 个（IDLE → INIT → ENUM → TRACE → CLEAR_SATB → POST_TRACE → PREFORWARD → FORWARD）

1.2 Go GC 架构（Concurrent Tri-color Mark-and-Sweep）

```
分配: TCMalloc风格 (mcache → mcentral → mheap, size-class)
          |
标记: Mark Setup (STW) → Concurrent Mark (并发, 混合写屏障) → Mark Termination (STW)
          |
清扫: Concurrent Sweep (并发, lazy sweep per-span)
```

核心特征：

- **非移动式**：对象分配后永不移动，通过清扫回收死亡对象的空间
- **无 tagged pointer**：不需要指针修正，无读屏障开销
- **仅写屏障**：混合写屏障（Dijkstra 插入 + Yuasa 删除），**无读屏障**
- **3 个 Phase**：Off → Mark → Mark Termination (→ Sweep 是并发不算 phase)
- **亚毫秒 STW**：仅 Mark Setup 和 Mark Termination 两个极短 STW

1.3 关键差异对比

维度	当前 Cangjie GC	Go GC	迁移影响
对象移动	移动（Copy）	不移动	删除 Forward 阶段，增加 Sweep
碎片处理	复制压缩消除	靠 size-class 和 span 回收缓解	需改造分配器
指针语义	Tagged Pointer + 自愈	原始指针	删除 tag 机制，大幅简化
读屏障	有（Forward 阶段修正）	无	删除读屏障，提升 mutator 吞吐
写屏障	SATB（Trace 阶段）	混合写屏障（Mark 阶段）	改造写屏障实现
STW 次数	多点（Enum/Preforward 有同步窗口）	2 次极短 STW	简化同步
GC Phase	8 个	3 个	大幅简化状态机
大对象	独立 Region, 标记清除	独立 span, 标记清除	基本一致

1.4 迁移动机

1. **降低 STW 延迟**：消除 Forward 阶段的同步窗口，仅保留两个亚毫秒级 STW
2. **移除读屏障**：Go 无读屏障，mutator 热路径零额外开销
3. **简化指针语义**：去掉 tagged pointer 和 forwarding table，降低调试和维护复杂度
4. **减少 Phase 数量**：从 8 个减到 3 个，状态机更简洁
5. **提升吞吐量**：非移动式 GC 省去了大量对象复制开销（尤其大堆场景）

代价：

- 无内存压缩，可能产生碎片（通过 size-class 分配器缓解）
- 清扫阶段可能延迟内存释放（通过 lazy sweep 和后台 sweep 缓解）

第二部分：迁移总体策略

2.1 核心原则

1. **渐进式迁移**：通过编译宏 `MRT_USE_MARK_SWEEP`（与现有 `MRT_USE_COPYGC` 对偶）控制新旧 GC 切换，确保任何一步都可回退
2. **复用现有框架**：保留 `RegionSpace`/`RegionManager` 内存管理底座，保留 `GCThreadPool` 并行框架，保留 `MutatorManager` 同步机制
3. **自底向上**：先改分配器，再改标记，再加清扫，最后删老代码
4. **验证驱动**：每一步都必须通过现有测试用例 + 新增专项测试

2.2 迁移路线图

```
Step 0: 构建可切换 GC 框架            ← 基础设施
    |
Step 1: 改造分配器（bitmap 模式）       ← 为非移动式 GC 做准备
    |   ├─ Step1.1: MSRegionInfo 元数据（64B）
    |   ├─ Step1.2: SizeClass + AllocBufferMS + CentralRegionPool
    |   └─ Step1.3: 大对象分配 + 堆增长
    |
Step 2: 实现三色标记                   ← 核心算法替换
    |   ├─ Step2.1: MarkSetup（Handshake）
    |   ├─ Step2.2: ConcurrentMark（复用 DoTracing）
    |   ├─ Step2.3: MarkTermination（STW 收敛）
    |   └─ Step2.4: PostMarkTermination（非 STW 清理）
    |
Step 3: 实现 Go 风格混合写屏障         ← 屏障替换
    |   ├─ Step3.1: MSBarrier（2 套替代 6 套）
    |   ├─ Step3.2: WriteBarrierBuffer（per-thread WBBuf）
    |   ├─ Step3.3: 读屏障 + TaggedPointer 移除
    |   └─ Step3.4: 对 Step2 各子阶段的改造
    |
Step 4: 实现并发清扫                   ← 并行回收
    |   ├─ Step4.1: SweepRegion（bitmap NOT 算法）
    |   ├─ Step4.2: Sweep 调度 + CentralRegionPool 集成
    |   └─ Step4.3: 后台 Sweep + FinalizerProcessor 集成
    |
Step 5: 实现 GC Pacer                  ← 智能触发控制
    |
Step 6: 移除 Forward 相关代码          ← 清理旧路径
    |
Step 7: 性能调优与压力验证             ← 收尾
```

2.3 关键文件变更范围预估

模块	变更类型	关键文件
Collector 框架	新增	`src/Heap/Collector/MarkSweepCollector.h/cpp`
Barrier	新增	`src/Heap/MSBarrier/MSIdleBarrier.h/cpp`、`MSWriteBarrier.h/cpp`、`WriteBarrierBuffer.h`

模块	变更类型	关键文件
Allocator	新增+改造	src/Heap/Allocator/MSRegionInfo.h、AllocBufferMS.h/cpp、CentralRegionPool.h/cpp、SizeClass.h
大对象	新增	src/Heap/Allocator/LargeObjectSpace.h/cpp
Heap 入口	改造	src/Heap/Heap.cpp、src/Heap/Heap.h
CompilerCalls	改造	src/CompilerCalls.cpp（写屏障入口，读屏障降级为直接 load）
FinalizerProcessor	改造	src/Heap/FinalizerProcessor.cpp（新增 RunMS() 后台 Sweep 循环）
TypeDef	改造	src/Heap/TypeDef.h（新增 MS_PHASE_* 枚举 20-24）
CMake	改造	CMakeLists.txt、config.cmake

2.4 兼容性承诺

- 新旧 GC 通过编译宏完全隔离，构建时选择
- 公共 API（Cangjie.h）保持不变
- 编译器侧 intrinsic 接口（llvm.cj.*）保持不变
- 对象模型（GCTib、TypeInfo）保持不变

第三部分：分步实施计划

Step 0：前置准备——构建可切换的 GC 框架

目标

在不影响现有 Copy GC 的前提下，搭建 Mark-Sweep GC 的编译开关和骨架代码。

0.1 新增编译宏

文件: config.cmake

```
# 新增 Mark-Sweep GC 开关（与 COPYGC_FLAG 互斥）
option(MARKSWEEP_FLAG "Enable Mark-Sweep GC (Go-style)" OFF)

if(MARKSWEEP_FLAG AND COPYGC_FLAG)
    message(FATAL_ERROR "COPYGC_FLAG and MARKSWEEP_FLAG are mutually exclusive")
endif()

if(MARKSWEEP_FLAG)
    add_definitions(-DMRT_USE_MARK_SWEEP=1)
endif()
```

文件: build.py

```
# 新增命令行参数
parser.add_argument('--gc-mode', choices=['copygc', 'marksweep'], default='copygc',
                    help='Select GC algorithm')
```

0.2 新建 Collector 骨架

新建文件: `src/Heap/Collector/MarkSweepCollector.h`

```
#ifndef CANGJIE_RUNTIME_MARK_SWEEP_COLLECTOR_H
#define CANGJIE_RUNTIME_MARK_SWEEP_COLLECTOR_H

#include "TracingCollector.h"

namespace cj::gc {

// Go 风格 GC 的 Phase 定义
enum class MSGCPhase : uint8_t {
    OFF = 0,          // GC 未运行，等价于 IDLE
    MARK_SETUP,       // STW: 开启写屏障，扫描根
    MARK,             // 并发标记
    MARK_TERMINATION,// STW: 完成标记，关闭写屏障
    SWEEP,            // 并发清扫
};

class MarkSweepCollector : public TracingCollector {
public:
    void DoGarbageCollection() override;

private:
    // Phase 1: Mark Setup (STW)
    void MarkSetup();

    // Phase 2: Concurrent Mark
    void ConcurrentMark();

    // Phase 3: Mark Termination (STW)
    void MarkTermination();

    // Phase 4: Concurrent Sweep
    void ConcurrentSweep();

    // GC Pacer
    void UpdateGCTrigger();

    MSGCPhase currentPhase_ = MSGCPhase::OFF;
};

} // namespace cj::gc

#endif
```

0.3 CollectorProxy 适配

文件: `src/Heap/Collector/CollectorProxy.h/cpp`

在 `CollectorProxy` 初始化时根据宏选择 collector：

```
void CollectorProxy::Initialize() {
#if defined(MRT_USE_MARK_SWEEP)
    currentCollector_ = new MarkSweepCollector();
#else
    currentCollector_ = new WCollector();  // 现有 Copy GC
#endif
}
```

0.4 验证标准

- `MRT_USE_MARK_SWEEP=0` 时，行为与改造前完全一致
- `MRT_USE_MARK_SWEEP=1` 时，可编译通过（骨架代码，GC 暂不执行实际回收）
- 现有全部测试用例通过

Step 1：改造分配器——从 bump-pointer 到 bitmap 分配模式

> **详细设计**：Step1_分配器改造详细设计.md **子文档**：Step1.1 MSRegionInfo 元数据 · Step1.2 SizeClass 与线程本地分配 · Step1.3 大对象与堆增长

目标

在非移动式 GC 下，对象分配后不再搬迁。改造 Region 内部分配策略，用 **bitmap + CTZ** 替代 bump-pointer，支持对象原地回收后空间复用。

1.1 核心设计：Bitmap 分配（非 free-list）

不采用 free-list（需触碰死对象内存，cache 不友好），改用 bitmap 位图管理空闲 slot：

```
Region (32KB)
├── allocBitmap[8 × uint64_t]   ← 1 = 空闲, 0 = 已占用
├── markBitmap[8 × uint64_t]    ← GC 标记结果
└── object slots (512 × 64B)    ← 以 sizeClass=64B 为例
```

- **分配**：`idx = CTZ(allocBitmap[word])` — O(1) 找到第一个空闲 slot
- **Sweep**：`allocBitmap[i] = ~markBitmap[i]` — 1 条 NOT 指令 = 64 slot 的清扫

1.2 Size Class（36 级，8B ~ 16KB）

```
// 新增文件: src/Heap/Allocator/SizeClass.h
static constexpr size_t kNumSizeClasses = 36;
// 8, 16, 24, 32, 48, 64, 80, 96, 112, 128,
// 160, 192, 224, 256, 320, 384, 448, 512,
// 640, 768, 896, 1024, 1280, 1536, 1792, 2048,
// 2560, 3072, 3584, 4096, 5120, 6144, 8192, 10240,
// 12288, 16384

// O(1) 查表映射
inline uint8_t SizeToClass(size_t size);
inline size_t ClassToSize(uint8_t cls);
```

1.3 MSRegionInfo（64 字节，替代现有 ~120B 元数据）

```
struct MSRegionInfo {  // 64 bytes, cache-line 对齐
    uint64_t*   allocBitmap_;    // 空闲位图 (1=可分配)
    uint64_t*   markBitmap_;     // GC 标记位图
    uintptr_t   dataStart_;      // 数据区起始地址
    uint16_t    totalSlotCount_; // 总 slot 数
    uint16_t    liveSlotCount_;  // 存活 slot 数
    uint16_t    freeSlotCount_;  // 空闲 slot 数
    uint8_t     sizeClass_;      // 所属 size class
    uint8_t     sweepState_;     // UNSWEPT(0)/SWEEPING(1)/SWEPT(2)
    uint32_t    sweepGen_;       // 清扫代数 (与全局 +2/GC 对比)
    // ... 链表指针等
};
```

关键改动：删除 CopyGC 专有字段（routeInfo、Ghost 备份、rwLock 约 56B），新增 bitmap 指针和 sweep 状态。

1.4 AllocBufferMS（线程本地缓存，对应 Go mcache）

```
class AllocBufferMS {
    // 每个 size class 维护一个当前活跃 Region
    MSRegionInfo* activeRegions_[kNumSizeClasses];  // 36 个槽位

    // 快速路径 (内联)
    uintptr_t FastAlloc(uint8_t cls) {
        auto* r = activeRegions_[cls];
        if (r && r->freeSlotCount_ > 0) {
            return r->AllocFromBitmap();  // CTZ 找空闲 slot
        }
```

```
        return SlowAlloc(cls);  // 从 CentralRegionPool 补充
    }
};
```

编译器约束兼容：`AllocBuffer::tlRegion` 偏移 0、`allocPtr` 偏移 0（JIT 硬编码依赖）。

1.5 CentralRegionPool（全局池，对应 Go mcentral）

每个 size class 维护 **4 条链表**（非旧版的 2 条）：

```
struct PerClassBucket {
    MutexLock lock;
    RegionList partialSwept;     // 已 sweep、有空闲 → 最优先
    RegionList fullSwept;        // 已 sweep、全满
    RegionList partialUnswept;   // 未 sweep、有空闲
    RegionList fullUnswept;      // 未 sweep、全满
};
CentralRegionPool::buckets_[kNumSizeClasses];  // 36 桶
```

分配优先级：`partialSwept → partialUnswept(原地sweep) → fullUnswept(原地sweep) → 新建 Region`。

1.6 大对象分级（≥ 16KB）

分级	范围	策略
Tier 0	≤ 16KB	bitmap 分配（走 size class）
Tier 1	16KB ~ 4MB	`LargeObjectSpace` + CartesianTree 低地址优先
Tier 2	> 4MB	独立 `mmap/munmap`，不进 Region 体系

1.7 关键设计决策

决策	选择	理由
空间复用方式	allocBitmap + markBitmap（1 bit/slot）	比 free-list 快，不触碰死对象内存
Size class 数量	36 级	覆盖 8B~16KB，内部碎片 ≤ 25%
Region 大小	**32KB**（非 128KB）	减少内碎片，加速 sweep
大对象阈值	**16KB**（非 32KB）	与 32KB Region 匹配
新 Region 分配	仍用 bump-pointer	sweep 后才切换 bitmap
堆镜像	保留，粒度改为 per-Region(32KB)	O(1) 查找
Pinned 对象	保持不变	MS 下所有对象都不移动

1.8 验证标准

指标	合格标准
单线程分配吞吐	≥ Copy GC 90%
多线程(8)分配吞吐	≥ Copy GC 85%
bitmap 分配 P99	≤ 200ns
sweep 后 Region 复用率	新增 Region 减少 ≥ 40%

Step 2：实现三色标记——替换 Forward 为 Mark-Sweep

> **详细设计**：Step2_三色标记与Sweep详细设计.md **子文档**：Step2.1 MarkSetup · Step2.2 ConcurrentMark · Step2.3 MarkTermination · Step2.4 PostMarkTermination

目标

实现 `MarkSweepCollector`（继承 `TracingCollector`），用 Go 风格并发三色标记替换 Trace + Forward（复制式）回收。Sweep 设计见 Step 4。

2.1 三色抽象与 Bitmap 映射

颜色	实现
白色 (White)	`RegionBitmap::IsMarked(offset) == false`
灰色 (Gray)	bitmap=1 **且** 在 WorkStack 中
黑色 (Black)	bitmap=1 **且** 已出队处理完毕

bitmap 使用现有 `RegionBitmap`：1 bit = 8 bytes，`atomic<uint64_t> markWords[]`，CAS 并发标记。

2.2 GC Phase 状态机（5 状态，枚举值 20-24）

```
MS_PHASE_IDLE (20)
  → MS_PHASE_MARK_SETUP (21)
  → MS_PHASE_MARK (22)
  → MS_PHASE_MARK_TERMINATION (23)
  → MS_PHASE_POST_MARK (24)
  → MS_PHASE_IDLE (20)
```

注意：不使用 Go 的 mark bit flip 技巧。每轮 GC 通过 `ClearAllMarkBitmaps()` 清零，更简单可靠。

2.3 四阶段核心流程

```
void MarkSweepCollector::DoGarbageCollection() {
    // Phase 1: MarkSetup (Handshake, 非全量 STW)
    MarkSetup();
    // Phase 2: Concurrent Mark (并发)
    ConcurrentMark();
    // Phase 3: MarkTermination (STW)
    MarkTermination();
    // Phase 4: PostMarkTermination (非 STW)
    PostMarkTermination();
    // Sweep 由 Step4 的三路协作异步完成
}
```

2.4 Phase 1: MarkSetup（Handshake 方式，目标 < 100μs）

```
MarkSetup():
  ① ForceFinishSweep()          // 确保上轮 sweep 全部完成
  ② RetireAllTLRegions()        // 刷回线程本地 Region
  ③ ClearAllMarkBitmaps()       // 两阶段清零（先小对象 Region，再大对象）
  ④ InstallWriteBarrier()       // SATB 删除屏障（Step2 初版）/ 混合屏障（Step3 后）
  ⑤ EnumAndMarkRoots()          // 复用 EnumAllCommonRoots
  ⑥ EnableMarkNewObject()       // Mark 期间新分配对象直接标黑
  ⑦ sweepGen += 2; gcEpoch++    // 推进代数
```

关键复用：根扫描复用 `TracingCollector::EnumAllCommonRoots()`，safepoint + `GcPhaseEnum` 机制不变。

2.5 Phase 2: ConcurrentMark（并发标记）

复用现有 `ConcurrentMarkingWork::Execute` 主循环：`pop → MarkObject → TraceRefFields → TryForkTask`。

新增特性：

- **Mark Assist**：分配债务驱动 mutator 辅助标记
- **新分配对象标黑**：Mark 期间 Allocate 路径中 `MarkObject(addr)`
- **收敛判定**：全局 + 局部队列空，连续 N 轮稳定

2.6 Phase 3: MarkTermination（STW）

```
MarkTermination():
  ① 收集 SATB/Extern/Finalizer → 主队列
```

```
② Execute 同构循环补标（排空残留灰色对象）
③ TransitionToGCPhase(MS_PHASE_POST_MARK)
```

2.7 Phase 4: PostMarkTermination（非 STW）

```
PostMarkTermination():
  ① ClearWeakRefBuffer()        // 弱引用清理
  ② ClearSatbBuffer()           // 清空写屏障 buffer
  ③ ProcessFinalizers()         // 析构器处理
  ④ PrepareCycleRef() [OHOS]    // 循环引用（仅 OHOS）
  ⑤ CollectLargeGarbage()       // 大对象 Eager Sweep
  ⑥ PrepareForSweep() + NotifyToSweep()  // 启动 Step4 并发清扫
```

2.8 仅需 Override 3 个方法

方法	WCollector 行数	MS 简化后	核心变化
`EnumRefFieldRoot`	~50 行	~5 行	去除 tag/forward/CAS
`EnumAndTagRawRoot`	~40 行	~5 行	去除 tag/CAS
`TraceRefField`	~40 行	~5 行	去除 tag/Forward/CAS

2.9 废弃组件（由 `#ifdef MRT_USE_MARK_SWEEP` 排除）

`ForwardObject`、`ForwardDataManager`、`TagID`/`FlipTagID`、`RefineFromSpace`、`Preforward*`、`GhostFromRegion`、`RouteInfo`/`RouteState`、`CopyCollector` 基类

2.10 关键设计决策

决策	选择	理由
类继承	`MarkSweepCollector : TracingCollector`	复用标记框架，不需 Forward/Copy
写屏障（Step2 初版）	SATB 删除屏障 + 新分配标黑	简单可靠，与 Step3 分步迁移
Mark Bit 机制	每轮 ClearBitmap（非 bit flip）	更简单，不引入 Go 指针高位 flip
根枚举	Handshake 方式	保守方案，后续可优化
ConcurrentMarkingWork	直接复用	内部通过 virtual 注入差异

2.11 验证标准

级别	内容
L1 单元	ClearAllMarkBitmaps / MSTraceBarrier / MarkNewObject / WorkStack split
L2 集成	gc_minimal.cj / gc_large.cj / gc_mixed.cj / WeakRef / Finalizer
L3 压力	16 协程 × 100000 次分配，长期引用不误回收
L4 对比	MS STW < WCollector 总 STW；堆占用 ↓50%（无 to-space）
L5 Sanitizer	ASAN + TSAN 零错误

Step 3：实现 Go 风格混合写屏障

> **详细设计**：Step3_混合写屏障详细设计.md **子文档**：Step3.1 MSBarrier 实现 · Step3.2 WBBuf 设计 · Step3.3 读屏障移除 · Step3.4 对 Step2 子阶段改造

目标

用 2 套 Barrier（MSIdleBarrier + MSWriteBarrier）替代现有 6 套屏障体系，实现**零读屏障**并发标记，彻底移除 Tagged Pointer。

3.1 现有屏障体系（6 套，被替换）

类	Phase	读屏障	写屏障

类	Phase	读屏障	写屏障
IdleBarrier	IDLE	tag 检查 + CAS	直接写
EnumBarrier	ENUM	tag + forward	SATB 记旧+新值
TraceBarrier	TRACE	tag + forward	SATB 仅记旧值
PostTraceBarrier	POST_TRACE	tag + forward	SATB + tag
PreforwardBarrier	PREFORWARD	forward + tag	forward 旧值
ForwardBarrier	FORWARD	**循环 TryForwardRefField**	直接写

3.2 新 Barrier 体系（2 套替代 6 套）

类	Phase	读	写
MSIdleBarrier	IDLE / POST_MARK / SWEEP	直接 load（零开销）	直接 store
MSWriteBarrier	MARK_SETUP / MARK / MARK_TERMINATION	直接 load（零开销）	shade(old) + shade(new) → fence → store

3.3 混合写屏障核心算法

```
writePointer(slot, ptr):       // 仅对堆上的写
    shade(*slot)                // Yuasa 删除屏障：旧值标灰
    shade(ptr)                  // Dijkstra 插入屏障：新值标灰（无条件）
    *slot = ptr
```

与旧版的关键修正：`shade(new)` 是**无条件**的（非"仅当已扫描栈"），避免 per-thread 扫描状态判断。

栈不需要屏障的原因：

1. MarkSetup STW 扫描所有栈（根全部标灰）
2. 堆上每次赋值 old+new 都 shade
3. 新分配对象直接标黑

3.4 WriteBarrierBuffer（per-thread，256 entries / 2KB）

```
class WriteBarrierBuffer {
    static constexpr size_t kBufferSize = 256;
    uintptr_t buf_[kBufferSize];  // 内嵌 Mutator，无动态分配
    size_t pos_ = 0;

    void Push(uintptr_t obj) {    // ~2 条指令（无锁）
        buf_[pos_++] = obj;
        if (pos_ >= kBufferSize) Flush();
    }
    void Flush();  // 批量转移到 per-P localWorkBuf
};
```

替代现有 `SatbBuffer::Node`（~512B，全局 arena，retire 链表）。

3.5 MSWriteBarrier 各方法行为

方法	行为
`WriteReference`	shade(old) + shade(new) → release fence → store
`WriteStaticRef`	同上（修正旧 TraceBarrier 不对静态引用记 SATB 的遗漏）
`WriteStruct`	遍历 old refs shade → memcpy → 遍历 new refs shade
`CopyRefArray`	逐元素 shade(old) + shade(new) → bulk memcpy
`AtomicSwapReference`	shade(new) → exchange → shade(old)
`CompareAndSwapReference`	shade(old) + shade(new)（无论 CAS 结果）

方法	行为
所有 Read*	直接返回 `field.GetTargetObject()`，**零开销**

3.6 读屏障 + Tagged Pointer 移除

- **7 个 CompilerCalls 读入口**（`CJ_MCC_ReadRefField` 等）降级为直接 load
- **`RefField`** 简化为裸指针：`GetTargetObject()` = 直接 load
- **清理**：`FlipTagID`、`IsOldPointer`、`IsCurrentPointer`、`TryUpdateRefField`
- **约束**：所有 `extern "C"` 函数签名完全不变

```
#ifdef MRT_USE_MARK_SWEEP
    #define REF_TO_ADDR(ref) (ref)
    #define ADDR_TO_REF(addr) (addr)
#else
    // 现有 Copy GC 的 tag 逻辑保留
#endif
```

3.7 对 Step2 子阶段的改造影响

子阶段	影响级别	关键变更
Step2.1 MarkSetup	中	Barrier 从 MSTraceBarrier → MSWriteBarrier
Step2.2 ConcurrentMark	**高**	WBBuf flush 实时写入 per-P localWorkBuf
Step2.3 MarkTermination	中	CollectSatbCandidates → FlushAllWriteBarrierBuffers
Step2.4 PostMarkTermination	低	ClearSatbBuffer → ClearWriteBarrierBuffers

3.8 性能预估

维度	预估
读操作吞吐	提升 **15-30%**（零额外指令 vs tag 检查）
写操作	持平或略降（多 shade(new) 但省 tag + seq_cst fence）
浮动垃圾	增加 5-15%
代码净减少	**~1150 行**（删 ~1950 行旧 Barrier，新增 ~800 行）

3.9 关键设计决策

决策	选择	理由
屏障套数	2 套 (Off/On)	替代 6 套，简化维护
栈屏障	不需要	MarkSetup STW 扫描所有栈 + 堆混合屏障
shade(new)	**无条件**	避免 per-thread 扫描状态判断
seq_cst fence	不需要	relaxed/release 即可
Sweep 阶段屏障	不需要	phase 已切 IDLE

3.10 验证标准

测试	关键点
TestMSWriteBarrier_ShadeOldAndNew	old+new 均被标灰
TestWBBuf_FlushOnFull	256 entries 满时自动 flush
TestRefField_MSMode_NoTag	MS 模式 RefField 为纯裸指针
TestConcurrentWriteBarrier_Stress	8 Mutator + 1 GC，10 秒无 use-after-free
TestMS_HybridBarrier_FullGC	10000 对象 + 5000 次修改，漏标率=0%

Step 4：实现并发清扫（Concurrent Sweep）

> **详细设计**：Step4_并发清扫详细设计.md **子文档**：Step4.1 SweepRegion 实现 · Step4.2 Sweep 调度与 CentralRegionPool · Step4.3 后台 Sweep 与 FinalizerProcessor

目标

将 markBitmap 翻转为 allocBitmap（`allocBitmap[i] = ~markBitmap[i]`），实现零 STW 并发清扫。**不新增独立 Sweeper 类**，由三路协作完成。

4.1 核心等式

```
allocBitmap[i] = ~markBitmap[i]    // 1 条 NOT 指令 = 64 slots 的清扫
```

不使用 free-list 遍历（不触碰死对象内存），严格对齐 Step1 的 bitmap 设计。

4.2 三路协作模型

路径	名称	触发方	行为
A	**后台 Sweep**	gc-helper 线程 `BackgroundSweepLoop()`	主动遍历 36 个 sizeClass 的 Unswept 链表
B	**Lazy Sweep**	分配线程 `CentralRegionPool::GetRegion(cls)`	partialSwept 为空时从 Unswept 取出原地 sweep
C	**ForceFinishSweep**	下轮 GC MarkSetup	强制 sweep 所有残余，确保 markBitmap 全清零

4.3 SweepRegion 核心算法

```
void MSRegionInfo::SweepRegion() {
    size_t wordCount = (totalSlotCount + 63) / 64;
    uint16_t live = 0;
    for (size_t i = 0; i < wordCount; i++) {
        allocBitmap[i] = ~markBitmap[i];              // 1 NOT = 64 slots
        live += __builtin_popcountll(markBitmap[i]);
    }
    memset(markBitmap, 0, wordCount * sizeof(uint64_t));
    liveSlotCount = live;
    freeSlotCount = totalSlotCount - live;
    sweepState = MS_SWEEP_STATE_SWEPT;
}
```

复杂度：**O(N/64)**，N = slot 总数。

4.4 sweepGen 机制（对标 Go `mheap.sweepgen`）

```
region.sweepGen == sg - 2  →  未 sweep
region.sweepGen == sg - 1  →  正在被某线程 sweep（CAS 中间态）
region.sweepGen == sg      →  已 sweep
```

`globalSweepGen_` 为 `std::atomic<uint32_t>`，每轮 GC **+2**（非 +1）。代际翻转 O(1)，无需移动链表节点。

CAS 失败 → 跳过（不等待），对标 Go `mcentral.cacheSpan`。

4.5 CentralRegionPool 四链表 + Lazy Sweep 优先级

```
GetRegion(cls):
  ① partialSwept     → pop → 直接用（最快，无 sweep）
  ② partialUnswept   → pop → CAS sweep → 用
  ③ fullUnswept      → pop → CAS sweep → 全死Release/有空洞用/仍满放回
  ④ 全空             → GetFreshRegion → FreeRegionManager
```

4.6 Region 命运判定

条件	去向
`freeSlotCount == totalSlotCount`	`ReleaseRegion()` → FreeRegionManager（回收物理页）
`0 < freeSlotCount < totalSlotCount`	`partialSwept` 链表
`freeSlotCount == 0`	`fullSwept` 链表

4.7 gc-helper 线程集成（FinalizerProcessor 扩展）

```
FinalizerProcessor::RunMS() 主循环:
  ① ProcessFinalizables()       // 最优先（resurrected 对象先执行 finalizer）
  ② BackgroundSweepLoop()       // 新增：逐 sizeClass sweep
  ③ FeedHungryBuffers()
  ④ ReclaimHeapGarbage()
  → Wait()
```

关键约束：Finalizer **必须在 Sweep 之前**执行（resurrected 对象需先 finalize 再判定存活）。

4.8 大对象策略

大对象 Eager Sweep 由 Step2.4 `PostMarkTermination::CollectLargeGarbage()` 完成（检查 mark bit → 存活保留 / 死亡回收）。**Step4 仅处理小对象 Region**（36 个 size class）。

4.9 Phase 联动

- Sweep 发生在 `MS_PHASE_IDLE`，Barrier 为 MSIdleBarrier（零开销）
- `PrepareForSweep()` 在 PostMarkTermination 中调用（`sweepGen += 2`）
- `ForceFinishSweep()` 在下轮 MarkSetup 中调用

4.10 关键设计决策

决策	选择	理由
Sweep 方式	bitmap NOT（非 free-list）	不触碰死对象内存，O(N/64)
独立 Sweeper 类	**不新增**	集成到 CentralRegionPool + FinalizerProcessor
sweepGen	+2/GC（非 +1）	对标 Go，支持三态判定
队列结构	CentralRegionPool 四链表（非 AtomicRegionQueue）	复用 Step1 设计
大对象 Sweep	由 Step2.4 Eager 完成	不在 Step4 重复实现
CAS 失败策略	跳过不等待	对标 Go mcentral，减少竞争

4.11 修改清单（不新增独立文件）

文件	修改
`FinalizerProcessor.h/cpp`	新增 `RunMS()`、`BackgroundSweepLoop()`、`NotifyToSweep()`
`MarkSweepCollector.cpp`	PostMarkTermination 调用 `PrepareForSweep()` + `NotifyToSweep()`
`HeapManager.cpp`	GC 启动前调用 `ForceFinishSweep()`
`CMakeLists.txt`	无新增文件

4.12 验证标准

测试	关键点
TestSweepRegion_BitmapNOT	512 slot, 标记 60 存活, freeSlotCount=452
TestSweepGenCAS_8Threads	8 线程 100 Region, 每 Region 恰 sweep 1 次
TestForceFinishSweep_DrainAll	36×10=360 Region 全部清空
TestBackgroundAndLazy_Concurrent	1 gc-helper + 4 Mutator, 200 Region 协作完成
TestMS_MultiCycle_100Rounds	100 轮 GC, sweepGen 单调 +2, 堆稳定

性能指标	目标
STW 时间	不含 sweep
分配延迟 P99	≤ 10μs
Lazy sweep 触发比例	< 20%

Step 5：实现 GC Pacer（触发控制器）

目标

实现类似 Go 的 GC Pacer，智能决定何时触发下一次 GC，平衡内存使用和 GC 频率。

5.1 Go 的 Pacer 设计

Go 的 GC 触发由两个机制控制：

1. **GOGC**（默认 100）：堆增长到上次 GC 后存活量的 `(1 + GOGC/100)` 倍时触发
   - 例如上次 GC 后存活 100MB，GOGC=100 → 堆到 200MB 时触发
2. **GOMEMLIMIT**（Go 1.19+）：硬性内存上限，接近上限时更积极地 GC

Pacer 还有一个"GC CPU 目标"（默认 25%），会动态调节 mark assist 的工作量，使 GC 占用的 CPU 不超过目标值。

5.2 实现方案

```
class GCPacer {
public:
    // ====== 配置 ======

    // 对应 GOGC，默认 100（堆翻倍时触发）
    // 设为 -1 表示禁用自动 GC
    int32_t gcPercent_ = 100;

    // 对应 GOMEMLIMIT，默认 MaxInt64（无限制）
    size_t memoryLimit_ = SIZE_MAX;

    // GC CPU 目标占比（0.0~1.0），默认 0.25
    double gcCPUFraction_ = 0.25;

    // ====== 状态 ======

    // 上次 GC 后的存活字节数
    size_t lastLiveBytes_ = 0;

    // 触发阈值（字节）
    size_t triggerBytes_ = 0;

    // 当前已分配字节数
    std::atomic<size_t> allocatedBytes_{0};

    // ====== 核心逻辑 ======

    // GC 完成后调用，更新触发阈值
    void UpdateAfterGC(size_t liveBytes, size_t totalAllocated) {
        lastLiveBytes_ = liveBytes;

        // 基于 GOGC 的触发点
        size_t gcPercentTrigger = 0;
        if (gcPercent_ > 0) {
            gcPercentTrigger = liveBytes + liveBytes * gcPercent_ / 100;
        }

        // 基于 GOMEMLIMIT 的触发点（预留 sweep 和分配缓冲）
        size_t memLimitTrigger = memoryLimit_;
        if (memoryLimit_ != SIZE_MAX) {
            // 在 limit 的 95% 处触发，留出缓冲
            memLimitTrigger = memoryLimit_ * 95 / 100;
        }

        // 取较小值作为实际触发点
```

```
        triggerBytes_ = std::min(gcPercentTrigger, memLimitTrigger);

        // 重置分配计数器
        allocatedBytes_.store(0, std::memory_order_relaxed);
    }

    // 分配路径中调用，检查是否需要触发 GC
    bool ShouldTriggerGC(size_t allocSize) {
        size_t total = allocatedBytes_.fetch_add(allocSize, std::memory_order_relaxed) + allocSize;
        return total + lastLiveBytes_ >= triggerBytes_;
    }

    // 获取环境变量配置
    void InitFromEnv() {
        // 读取 CJ_GC_PERCENT（对应 GOGC）
        const char* gcPercent = getenv("CJ_GC_PERCENT");
        if (gcPercent) {
            gcPercent_ = atoi(gcPercent);
        }

        // 读取 CJ_MEM_LIMIT（对应 GOMEMLIMIT）
        const char* memLimit = getenv("CJ_MEM_LIMIT");
        if (memLimit) {
            memoryLimit_ = ParseSize(memLimit);
        }
    }
};
```

5.3 与现有触发机制的关系

现有机制	Pacer 中对应	处理方式
GC_REASON_HEU（启发式）	ShouldTriggerGC()	替换为 Pacer 逻辑
GC_REASON_OOM（内存不足）	保留	作为 fallback
GC_REASON_USER（手动触发）	保留	用户显式 GC
GC_REASON_BACKUP（兜底）	可保留	长时间无 GC 的安全网
GC_REASON_FORCE（强制）	保留	Runtime 内部强制 GC

5.4 Mark Assist（标记辅助）

Go 的一个重要机制：当 mutator 分配速度过快时，分配线程会被"征召"参与标记工作（mark assist），实现背压控制。

```
// 在分配路径中：
uintptr_t Allocate(size_t size) {
    if (IsInMarkPhase() && pacer_.NeedAssist(size)) {
        // 此线程需要帮助完成一些标记工作，再允许分配
        DoMarkAssist(size);
    }
    return DoAllocate(size);
}

void DoMarkAssist(size_t debtBytes) {
    // 从全局 workStack 取灰色对象并标记
    // 直到"偿还"了 debtBytes 对应的标记工作量
    size_t scanned = 0;
    while (scanned < debtBytes) {
        uintptr_t obj = workStack_.TryPop();
        if (obj == 0) break;
        MarkObject(obj);
        scanned += GetObjectSize(obj);
    }
}
```

这防止了"分配比标记快"导致的堆无限膨胀。

5.5 配置环境变量

```
# 设置 GC 触发比例（等价于 GOGC）
export CJ_GC_PERCENT=100    # 默认：堆翻倍触发
export CJ_GC_PERCENT=200    # 堆三倍触发（更少 GC，更多内存）
export CJ_GC_PERCENT=50     # 堆 1.5 倍触发（更频繁 GC，更少内存）

# 设置内存上限（等价于 GOMEMLIMIT）
export CJ_MEM_LIMIT=1G      # 限制堆不超过 1GB
export CJ_MEM_LIMIT=512M    # 限制堆不超过 512MB
```

5.6 验证标准

- 不同 `CJ_GC_PERCENT` 值下 GC 频率符合预期
- `CJ_MEM_LIMIT` 生效时堆大小不超限
- Mark Assist 能有效控制堆膨胀
- 高并发分配压力下不 OOM（Pacer 能及时触发 GC）

Step 6：移除 Forward 相关代码路径

目标

在 Mark-Sweep GC 稳定之后，清理 Copy GC 遗留的 Forward 相关代码（在 `MRT_USE_MARK_SWEEP` 宏保护下）。

6.1 可删除的模块/机制

模块	说明	影响文件
ForwardTable	对象路由映射表	`src/Heap/Collector/ForwardData.h/cpp`
GhostRegion	from→to Region 视图	`src/Heap/Allocator/GhostRegion.h/cpp`
Tagged Pointer	指针低位 tag 编码	`src/Heap/Allocator/RefField.h`, `TypeDef.h`
ForwardBarrier	Forward 阶段自愈屏障	`src/Heap/WCollector/ForwardBarrier.cpp`
PreforwardBarrier	Preforward 阶段屏障	`src/Heap/WCollector/PreforwardBarrier.cpp`
CopyObject	对象复制逻辑	`src/Heap/WCollector/WCollector.cpp`
RouteState 状态机	Region 搬迁状态	`src/Heap/Allocator/RegionInfo.h`
fromRegionList / unmovable	搬迁相关 Region 列表	`src/Heap/Allocator/RegionManager.cpp`
FlipTagID	Tag 交替翻转	`src/Heap/Collector/TracingCollector.cpp`

6.2 不可删除（Mark-Sweep 仍需复用）

模块	原因
`RegionInfo::MarkObject`	标记 bitmap 仍需使用
`TracingImpl` 标记主循环	并发标记复用
`EnumAllCommonRoots`	根扫描复用
`WeakRefBuffer`	弱引用处理复用
`FinalizerProcessor`	Finalizer 处理复用
`SatbBuffer` → 改为 `WriteBarrierBuffer`	缓冲思路复用，实现替换
`GCThreadPool`	并行框架复用
`MutatorManager`	同步框架复用

6.3 GC Phase 简化

```
// 删除（Copy GC 专有）：
//   GC_PHASE_INIT
//   GC_PHASE_CLEAR_SATB_BUFFER
//   GC_PHASE_POST_TRACE
//   GC_PHASE_PREFORWARD
```

```
//   GC_PHASE_FORWARD
//   GC_PHASE_RECLAIM_SATB_NODE

// Mark-Sweep 仅需:
enum class MSGCPhase : uint8_t {
    OFF = 0,
    MARK_SETUP = 1,
    MARK = 2,
    MARK_TERMINATION = 3,
    SWEEP = 4,
};
```

6.4 屏障简化

```
现有 6 套屏障:
  IdleBarrier → EnumBarrier → TraceBarrier → PostTraceBarrier → PreforwardBarrier → ForwardBarrier

Mark-Sweep 仅需 2 套:
  MSIdleBarrier (GC Off)
  MSWriteBarrier (GC Mark)
```

6.5 执行策略

1. 使用 `#ifdef MRT_USE_MARK_SWEEP` 条件编译隔离
2. 保留 Copy GC 代码直到 Mark-Sweep 完全验证通过
3. 最终可通过删除 `MRT_USE_COPYGC` 代码段彻底清理

6.6 验证标准

- 删除后编译无 warning/error
- 二进制体积减小（预期减少约 15-20%）
- 所有测试用例在 Mark-Sweep 模式下通过

Step 7：性能调优与验证

目标

完成全面的性能评估、压力测试和调优，确保 Mark-Sweep GC 满足生产要求。

7.1 性能基准测试矩阵

测试维度	测试场景	关键指标
分配吞吐	单线程/多线程大量小对象分配	ops/sec
GC 停顿	各种堆大小下的 STW 时间	p50/p99/max 停顿
GC 吞吐	GC 占用 CPU 比例	GC CPU%
内存效率	峰值 RSS vs 存活对象大小	内存放大系数
碎片率	长时间运行后的内存碎片	碎片比例
尾延迟	应用级 p99 延迟	μs
大对象	大量大对象分配回收	回收及时性

7.2 关键调优参数

```
# GC 触发比例（越小越频繁，内存越紧凑）
export CJ_GC_PERCENT=100    # 默认

# 内存上限
export CJ_MEM_LIMIT=1G

# Size class 配置（可调整）
export CJ_SIZE_CLASS_CONFIG=default
```

```
# 并行 mark 线程数
export CJ_GC_THREADS=4

# 后台 sweep 线程数
export CJ_SWEEP_THREADS=1
```

7.3 与 Copy GC 性能对比（预期）

指标	Copy GC	Mark-Sweep GC	说明
STW 停顿	~1-10ms	**< 500µs**	大幅降低
读屏障开销	有	**无**	热路径提速
写屏障开销	SATB	混合（略高）	可接受
内存碎片	无（压缩）	**有**	需以 size class 缓解
大堆 GC 吞吐	差（复制量大）	**好**	不复制
小堆 GC 吞吐	好（快速复制）	一般	看具体场景

7.4 碎片缓解策略

1. **Size Class**：减少内部碎片（浪费 ≤ 12.5%）
2. **Region 回收策略**：空 Region 及时归还 OS
3. **CJ_GC_PERCENT 调小**：更频繁 GC，减少碎片累积
4. **紧急压缩**（可选 future work）：碎片率过高时，对部分 Region 做局部压缩

7.5 压力测试用例

```
gc/gc.cj                    — 综合用例
gc/gc_large.cj              — 大对象
gc/gc_mixed.cj              — 混合大小
gc/gc_quick.cj              — 快速分配
gc/gc_tl_region_leak.cj     — 内存泄漏检测
gc/gc_shrink_verify.cj      — 堆收缩验证
gc/test_los_functional.cj   — LOS 功能验证
gc/test_los_perf.cj         — LOS 性能测试
```

每个用例需在 Mark-Sweep 模式下验证通过，并与 Copy GC 做性能对比。

7.6 监控与可观测性

复用现有 `MRT_REPORT` 日志框架，新增以下指标：

```
[MS-GC] cycle=N live=100MB allocated=95MB trigger=200MB
[MS-GC] mark_setup_stw=80us mark_time=15ms mark_term_stw=50us sweep_time=8ms
[MS-GC] regions: total=1024 swept=800 freed=300 fragmentation=12%
[MS-GC] pacer: gc_percent=100 cpu_fraction=0.18 assist_bytes=5MB
```

第四部分：风险评估与回退方案

4.1 主要风险

风险	影响	缓解措施
碎片导致 OOM	堆有空闲空间但无法分配	Size class + Region 级回收 + 紧急压缩兜底
STW 不够短	根集过大导致 Mark Setup 超时	增量根扫描 + 栈预编译
并发标记漏标	对象被误回收 → crash	写屏障形式化验证 + Fuzzing 测试
Sweep 延迟过大	内存释放不及时	增加后台 sweep 线程 + Lazy sweep 加速

风险	影响	缓解措施
OHOS 跨语言兼容	Export Root 机制未适配	保留 Export Root 完整逻辑
编译器 IR 不兼容	`llvm.cj.*` intrinsic 语义变化	保持接口不变，仅改运行时实现

4.2 回退策略

1. **编译时回退**：`MRT_USE_MARK_SWEEP=0`，完全回到 Copy GC，零风险
2. **运行时切换**（未来可选）：通过环境变量 `CJ_GC_MODE=copygc|marksweep` 运行时选择
3. **混合模式**（未来可选）：大部分对象走 Mark-Sweep，碎片率过高时对部分 Region 做 compaction

4.3 分步回退点

步骤	回退条件	回退动作
Step 1 (分配器)	性能退化 > 30%	回退到 bump-pointer
Step 2 (三色标记)	正确性问题	回退到 Copy GC 全路径
Step 3 (混合写屏障)	漏标问题	回退到 SATB
Step 4 (并发 Sweep)	碎片严重	加紧急压缩机制
Step 5 (Pacer)	触发不合理	回退到现有启发式

4.4 测试覆盖要求

每一步交付前：

1. 现有所有 `gc/*.cj` 测试通过
2. 新增对应 Step 的专项测试
3. 24 小时稳定性压力测试通过
4. ASAN/TSAN 检查通过

附录

A. Go GC 源码参考

文件	内容	参考价值
`runtime/mgc.go`	GC 主控制流	Mark Setup/Termination 流程
`runtime/mgcmark.go`	并发标记	工作窃取、mark assist
`runtime/mgcsweep.go`	并发清扫	Lazy sweep、后台 sweep
`runtime/mgcpacer.go`	Pacer	触发阈值计算
`runtime/mbarrier.go`	写屏障	混合写屏障实现
`runtime/mheap.go`	堆管理	Span/arena 管理
`runtime/mcache.go`	线程本地缓存	Per-P 分配
`runtime/mcentral.go`	Central 缓存	Size class 分发
`runtime/sizeclasses.go`	Size class 定义	67 个 class 的具体值

B. 当前 Cangjie GC 源码锚点

文件	内容
`src/Heap/Heap.cpp`	堆入口 + 屏障安装
`src/Heap/Collector/TracingCollector.cpp`	标记主循环（可复用）
`src/Heap/WCollector/WCollector.cpp`	`DoGarbageCollection` 主流程
`src/Heap/Allocator/RegionSpace.cpp`	分配主流程
`src/Heap/Allocator/RegionManager.cpp`	Region 管理 + TakeRegion
`src/Heap/Allocator/RegionInfo.h`	Region 元数据 + mark bitmap

文件	内容
src/CompilerCalls.cpp	编译器屏障入口
src/Heap/WCollector/ForwardBarrier.cpp	Forward 屏障（待删除）

C. 名词对照表

Go 概念	Cangjie 对应	说明
goroutine	CJThread	轻量级线程
P (Processor)	Processor/Monitor	调度实体
mspan	MSRegionInfo（64B）	Region 元数据，含 allocBitmap/markBitmap/sweepGen/sweepState
span.allocBits	allocBitmap_	空闲 slot 位图（1=可分配），CTZ 分配
span.gcmarkBits	markBitmap_	GC 标记位图
arena	堆虚拟空间（64GB 预留）	mmap MAP_NORESERVE + 256MB Segment 提交
mcache	AllocBufferMS	线程本地缓存，per-class activeRegions_[36]
mcentral	CentralRegionPool	全局按 size class 分发，**4 条链表/class**
mcentral.partial	partialSwept / partialUnswept	有空闲 slot 的 Region（已/未 sweep）
mcentral.full	fullSwept / fullUnswept	全满 Region（已/未 sweep）
mheap	RegionManager + FreeRegionManager	顶层堆管理
mheap.sweepgen	globalSweepGen_（uint32_t）	每轮 GC +2，三态判定
sizeclasses	SizeClassInfo（36 级，8B~16KB）	O(1) 查表映射
largeSpan	LargeObjectSpace	Tier1: CartesianTree; Tier2(>4MB): mmap
GOGC	CJ_GC_PERCENT	GC 触发比例
GOMEMLIMIT	CJ_MEM_LIMIT	内存上限
wbBuf	WriteBarrierBuffer（256 entries/2KB）	per-thread 写屏障缓冲，内嵌 Mutator
mark bit	RegionBitmap markWords[]	CAS 并发标记，1 bit = 8 bytes
gcWork	WorkStack（MarkStackBuf 64 元素/段）	灰色对象工作队列
mark assist	Mark Assist（分配债务驱动）	mutator 辅助标记
gcphase	MS_PHASE_*（20-24）	IDLE/MARK_SETUP/MARK/MARK_TERMINATION/POST_MARK
writeBarrier (off)	MSIdleBarrier	GC 不活跃时，读写均零开销
writeBarrier (on)	MSWriteBarrier	shade(old)+shade(new)，读零开销

D. 预估工作量

步骤	核心内容	预估工期	复杂度
Step 0: GC 框架	条件编译 + MarkSweepCollector 空壳	1 周	低
Step 1: 分配器改造	MSRegionInfo(64B) + bitmap(CTZ) + SizeClass(36) + AllocBufferMS + CentralRegionPool(4 链表) + LargeObjectSpace	3-4 周	**高**
Step 2: 三色标记	MarkSweepCollector:TracingCollector + 4 Phase(20-24) + Handshake MarkSetup + ClearBitmap + PostMarkTermination	3-4 周	高
Step 3: 混合写屏障	2 套 Barrier(MSIdleBarrier+MSWriteBarrier) + WBBuf(256) + 读屏障移除 + TaggedPointer 移除	2-3 周	高
Step 4: 并发 Sweep	bitmap NOT + sweepGen(+2/GC) + 三路协作 + FinalizerProcessor 集成（不新增类）	1-2 周	中
Step 5: GC Pacer	HeapGoalController + Mark Assist + CPU 预算	1-2 周	中

步骤	核心内容	预估工期	复杂度
Step 6: 清理旧代码	删除 Forward/Copy/Tag 全路径	1 周	低
Step 7: 性能调优	碎片分析 + STW 优化 + 压力测试	2-3 周	中
总计		**14-20 周**	